Otwartoźródłowe SDK do kontroli autonomicznego drona "wut-control"

Dokumentacja projektowa PZSP2

Wersja 2

30.10.2026

Semestr 2026Z

Zespół nr 1 w składzie:

Filip Baroniewski

Szymon Szczepanik

Piotr Kolimeczkow

Wojciech Kryński

Mentor zespołu: dr inż. Marcin Szlenk

Właściciel tematu: mgr inż. Piotr Walas

Spis treści

1 Wprowadzenie....2
1.1 Cel projektu....2
1.2 Wstępna wizja projektu....2
2 Metodologia wytwarzania....2
3 Analiza wymagań....2
3.1 Wymagania użytkownika i biznesowe....2
3.2 Wymagania funkcjonalne i niefunkcjonalne....2
3.3 Przypadki użycia....2
3.4 Potwierdzenie zgodności wymagań....2
4 Definicja architektury....3
5 Dane trwałe....3
5.1 Model logiczny danych....3
5.2 Przetwarzanie i przechowywanie danych....3
6 Specyfikacja analityczna i projektowa....3
7 Projekt standardu interfejsu użytkownika....3
8 Specyfikacja testów....3
9 Wirtualizacja/konteneryzacja....4
10 Bezpieczeństwo....4
11 Podręcznik użytkownika....4
12 Podręcznik administratora....4
13 Podsumowanie....4
14 Bibliografia....4

1 Wprowadzenie

1.1 Cel projektu

Celem projektu jest połączenie bibliotek do sterowania dronem w jedno wspólne SDK. Obecne biblioteki do drona Bitcraze są rozproszone na trzy osobne narzędzia, co utrudnia jednoczesne sterowanie, odczyt z sensorów i podgląd z kamery. Stworzone SDK ma ułatwić grupie badawczej trenowanie dronów i wykonywanie autonomicznych misji oraz ułatwić próg wejścia osobom początkującym bez konieczności dogłębnego poznania wymaganych bibliotek.

1.2 Wstępna wizja projektu

Ogólna wizja: Jedno narzędzie SDK o interfejsie w Pythonie z rdzeniem w C++, które z poziomu jednej klasy pozwala w kilku linijkach połączyć się z dronem, sprawdzić jego stan, odczytać lidary i kamerę oraz wysłać komendy lotu.

Użytkownicy: Grupa badawcza z sali 134, programiści oraz studenci trenujący algorytmy autonomiczne i badający modele.

Przetwarzane dane: Odczyty z lidarów, klatki obrazu z kamery oraz status telemetrii (poziom baterii, połączenie).

Tryb działania: Programista importuje bibliotekę (import wut_*control), wywołuje proste metody (np. go_left())* i uruchamia gotowe skrypty misji (np. omijanie przeszkód czy lot po punktach).

2 Metodologia wytwarzania

Role w zespole: Do ustalenia; Dwie wyznaczone osoby pełnią rolę opiekunów repozytorium z wyłącznym prawem do zatwierdzania i scalania kodu do głównej gałęzi.

Metodyka: Elastyczna praca w 1-tygodniowych cyklach z bieżącą tablicą zadań.

Wykorzystywane elementy:

> Kamienie milowe i zadania: Definiujemy wspólne cele nadrzędne, a osoba realizująca dany etap rozbija go na własne, szczegółowe zadania.
>
> Spotkania: Praca w cyklach tygodniowych podsumowywana krótkim spotkaniem, na którym omawiamy postępy i planujemy kolejny tydzień.
>
> Pull Requesty i Code Review: Cały kod trafia do repozytorium przez Pull Requesty. Warunkiem wdrożenia jest akceptacja przez co najmniej jedną uprawnioną osobę.

3 Analiza wymagań

3.1 Wymagania użytkownika i biznesowe

- wymagania biznesowe:

- Cel i potrzeba: Uproszczenie obsługi dronów w grupie badawczej oraz obniżenie progu wejścia dla nowych członków zespołu.
- Problem do rozwiązania: Rozproszenie oficjalnych bibliotek Bitcraze na trzy osobne narzędzia (cflib, MotionCommander, kamera)

- wymagania użytkowe:

- Potrzeby użytkowników: Możliwość sterowania dronem, podglądu parametrów i odczytu czujników za pomocą prostego interfejsu w Pythonie.
- Cechy użytkowe: Gotowe metody do obsługi drona, czytelna dokumentacja w stylu Read the Docs oraz gotowe skrypty przykładowych misji.

- wymagania systemowe:

- Architektura rozwiązania: Interfejs główny w Pythonie, połączony z rdzeniem napisanym w C++.
- Kluczowe funkcjonalności: Inicjalizacja drona, diagnostyka stanu, odczyt odległości z lidarów, dostęp do strumienia wideo z kamery oraz wysyłanie poleceń lotu.

3.2 Wymagania funkcjonalne i niefunkcjonalne

Wymagania funkcjonalne - podstawowe

- F1 (Inicjalizacja): Połączenie z dronem i przygotowanie go do pracy za pomocą pojedynczej metody.
- F2 (Health check): Weryfikacja stanu baterii, połączenia oraz gotowości sensorów przed startem.
- F3 (Sterowanie lotem): Wysyłanie prostych poleceń (start, lądowanie, lot do przodu/tyłu/na boki).
- F4 (Odczyt sensorów): Zwracanie bieżących odległości z lidarów.
- F5 (Podgląd z kamery): Udostępnianie obrazu z kamerki.

Wymagania funkcjonalne - dodatkowe

- F6 (Estymacja pozycji): Zwracanie orientacyjnej pozycji drona na podstawie danych z akcelerometru lub latarni, gdy system pozycjonowania zwraca 0,0,0.
- F7 (Obsługa pozycji relatywnej): pozycjonowanie względem latarni.
- F8 (Przykładowe misje): Dostarczenie gotowych skryptów wykorzystujących kamerę

oraz lidary m.in. omijanie przeszkód i szukanie markera.

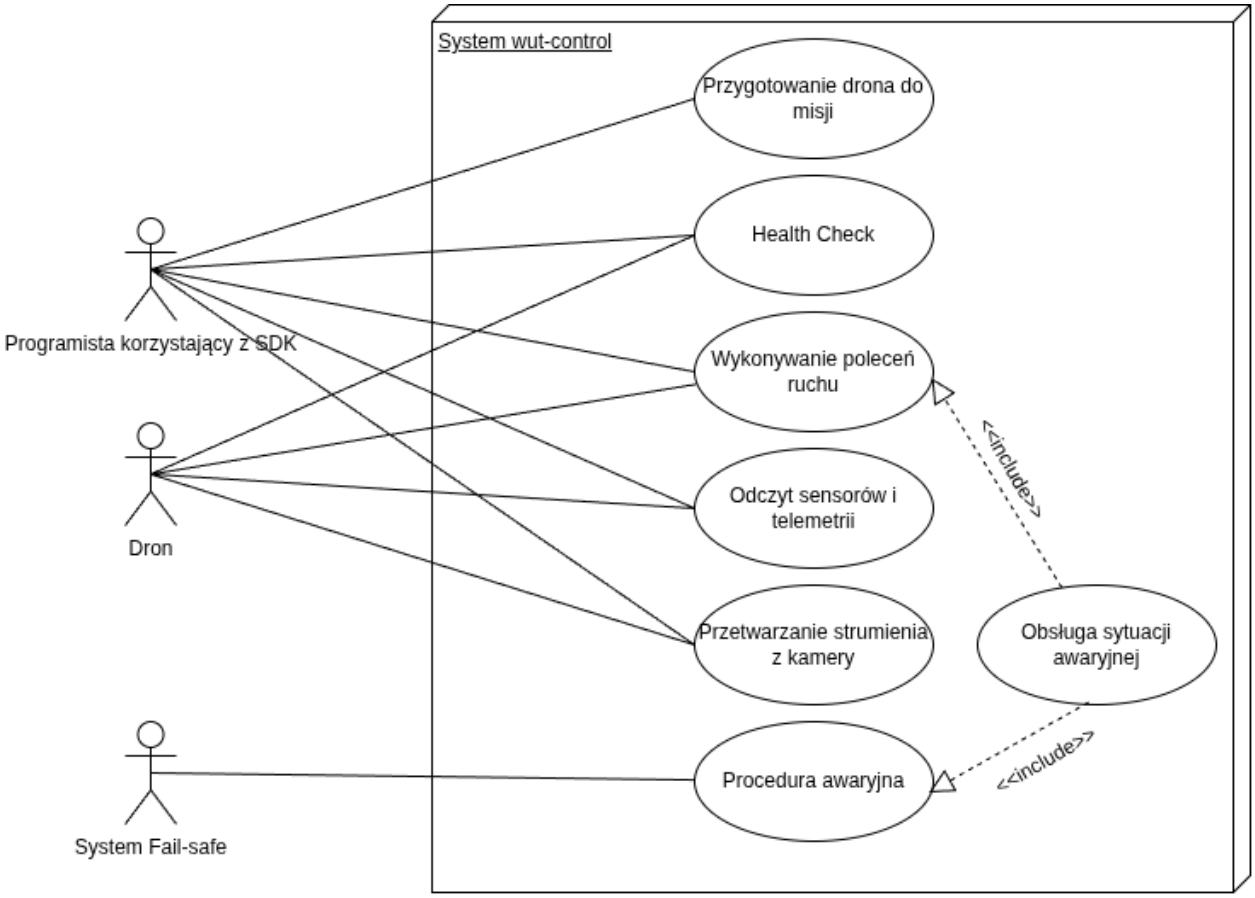
Wymagania niefunkcjonalne

- NF1 (Wydajność): Rdzeń w C++ zapewniający szybkie przetwarzanie obrazu i danych z sensorów bez opóźnień interpretera.
- NF2 (Prostota API): Minimalistyczny interfejs w Pythonie, wszystkie główne funkcje dostępne z poziomu jednej klasy.
- NF3 (Dokumentacja): Kompletny opis metod i przykłady użycia w standardzie Read the Docs.
- NF4 (Otwarty kod): Repozytorium udostępnione publicznie na GitHubie na licencji open source.

Wymaganie biznesowe / użytkowe	**Wymaganie systemowe**	**Wymaganie funkcjonalne**	**Wymaganie niefunkcjonalne**
Uproszczenie obsługi dronów w grupie badawczej oraz obniżenie progu wejścia dla nowych członków zespołu	Interfejs główny w Pythonie, połączony z rdzeniem napisanym w C++	**F1** (Inicjalizacja), **F3** (Sterowanie lotem)	**NF2** (Prostota API)
Rozproszenie oficjalnych bibliotek Bitcraze na trzy osobne narzędzia (cflib, MotionCommander, kamera)	Kluczowe funkcjonalności: Inicjalizacja drona, diagnostyka stanu, odczyt odległości z lidarów, dostęp do strumienia wideo z kamery oraz wysyłanie poleceń lotu	**F1** (Inicjalizacja), **F4** (Odczyt sensorów), **F5** (Podgląd z kamery)	**NF1** (Wydajność), **NF2** (Prostota API)
Możliwość sterowania dronem, podglądu	Interfejs główny w Pythonie	**F3** (Sterowanie	**NF2** (Prostota API)

parametrów i odczytu czujników za pomocą prostego interfejsu w Pythonie		lotem), **F4** (Odczyt sensorów), **F5** (Podgląd z kamery)	
Gotowe metody do obsługi drona	Diagnostyka stanu, wysyłanie poleceń lotu	**F2** (Health check), **F3** (Sterowanie lotem)	**NF1** (Wydajność)
Gotowe skrypty przykładowych misji	Zastosowanie odczytów z sensorów i kamery do zadań autonomicznych	**F8** (Przykładowe misje), **F6** (Estymacja pozycji), **F7** (Obsługa pozycji relatywnej)	**NF4** (Otwarty kod)
Czytelna dokumentacja w stylu Read the Docs	--	--	**NF3** (Dokumentacja)

3.3 Przypadki użycia



Powyższy diagram przedstawia główne przypadki użycia systemu wut-control. Wyróżniono trzech aktorów: programistę korzystającego z SDK, drona oraz system Fail-safe.

Programista korzystający z SDK może przygotować drona do misji, przeprowadzić kontrolę jego stanu, wydawać polecenia ruchu, odczytywać dane z sensorów i telemetrii oraz przetwarzać strumień obrazu z kamery.

Dron uczestniczy w realizacji poleceń sterujących oraz dostarcza dane niezbędne do monitorowania jego pracy. System Fail-safe odpowiada za obsługę sytuacji awaryjnych, takich jak utrata łączności lub wystąpienie błędu, poprzez uruchomienie odpowiedniej procedury awaryjnej.

3.4 Potwierdzenie zgodności wymagań

Należy wkleić wycinek ekranu pokazujący korespondencję z Mentorem i Właścicielem tematu.

4 Definicja architektury

[plan struktury systemu – model komponentów, modułów, serwisów

zastosowane szablony architektoniczne (np. MVC, mikroserwisy, SOA, szyna usług)

interfejsy kluczowych elementów struktury oraz wyjaśnienie połączeń i interakcji pomiędzy nimi

perspektywa logiczna/funkcjonalna oraz perspektywa sprzętowa (tzw. deployment): systemy operacyjne, serwer aplikacyjny, system bazy danych i inne mechanizmy utrwalania danych, system raportowania , system analityczny/BI, mechanizmy zarządzania, mechanizmy bezpieczeństwa

diagramy]

5 Dane trwałe

5.1 Model logiczny danych

[diagramy]

5.2 Przetwarzanie i przechowywanie danych

[opis planowanego sposobu implementacji: jaka baza danych lub inne oprogramowanie, jak będzie realizowany dostęp do danych (użyte API itp.), czy planowane jest programowanie w b.d. (wyzwalacze, mechanizmy optymalizacji zapytań, itp.)]

6 Specyfikacja analityczna i projektowa

Obowiązkowo odnośnik do repozytorium kodu (jedno repozytorium na projekt, jeżeli więcej proszę uzasadnić)

Obowiązkowo określenie metod realizacji: języki programowania, frameworki, środowisko programowania/ uruchamiania/ wdrażania, środowisko ciągłej integracji]

Obowiązkowo Diagram klas lub model pojęciowy struktury informacyjnej: E-R l

Opcjonalnie model struktury systemu (diagram wdrożenia)

Opcjonalnie specyfikacja realizacji przypadków użycia: diagramy sekwencji lub współpracy

Obowiązkowo statystyki: licaba plików, linie kodu, liczba testów jednostkowych

7 Projekt standardu interfejsu użytkownika

[wykorzystanie narzędzi do modelowania oraz tworzenia makiet warstwy prezentacyjnej (np. storyboards, wireframes, wireflows, mockups, prototypes etc)]

8 Specyfikacja testów

[standardy obsługi błędów i sytuacji wyjątkowych

rodzaje testów, specyfikacja i opis sposobu realizacji poszczególnych rodzajów testów, scenariusze testowe

miary jakości testów]

9 Wirtualizacja/konteneryzacja

10 Bezpieczeństwo

11 Podręcznik użytkownika

[instrukcja użycia funkcjonalności systemu]

12 Podręcznik administratora

[- instrukcja budowy systemu z kodu źródłowego

- instrukcja instalacji i konfiguracji systemu

- instrukcja aktualizacji oprogramowania

- instrukcja zarządzania użytkownikami i uprawnieniami

- instrukcja tworzenia kopii zapasowych i odtwarzania systemu

- instrukcja zarządzania zasobami systemu]

13 Podsumowanie

[Krytyczna analiza osiągniętych wyników, mocne i słabe strony

Możliwe kierunki rozwoju]

14 Bibliografia

[Wykaz materiałów źródłowych, opis zgodny ze standardem sporządzania opisów bibliograficznych - https://bg.pw.edu.pl/index.php/przypisy-i-bibliografia]

Zatwierdzam dokumentację.	…… ……. Data i podpis Mentora